🍷 Wine Quality Prediction Using Machine Learning

A machine learning project that predicts wine quality scores using physicochemical properties. Built with Python, Pandas, NumPy, Matplotlib, Seaborn, and Scikit-learn.

📌 Project Overview

The objective is to predict the quality score of wine based on its chemical properties. The project follows a complete machine learning workflow, including data inspection, exploratory data analysis, preprocessing, model training, evaluation, feature importance analysis, and hyperparameter tuning.

🎯 Objectives

- Explore and understand the WineQT dataset.
- Validate data types and inspect missing values, duplicates, and potential outliers.
- Analyze relationships between chemical properties and wine quality.
- Train and compare multiple classification algorithms.
- Evaluate model performance using classification metrics.
- Tune the Random Forest model using cross-validation.
- Build a reusable prediction function.

📂 Dataset

The project uses `WineQT.csv`.

- Initial records: 1,143
- Initial columns: 12
- Input features after removing `Id`: 11
- Target variable: `quality`
- Missing values reported during initial inspection: 0

Input Features

1. Fixed acidity
2. Volatile acidity
3. Citric acid
4. Residual sugar
5. Chlorides
6. Free sulfur dioxide
7. Total sulfur dioxide
8. Density
9. pH

10. Sulphates
11. Alcohol

Target: Wine quality score.

The `quality` column is kept separate from the input features to prevent target leakage.

🧰 Technologies Used

- Python
- Pandas and NumPy
- Matplotlib and Seaborn
- Scikit-learn
- Jupyter Notebook

⚙️ Workflow

1. Load and inspect the dataset.
2. Remove the identifier column and validate numeric data.
3. Explore quality-score distributions and chemical properties.
4. Analyze correlations and potential outliers.
5. Separate input features (`X`) and target (`y`).
6. Split the data using an 80/20 stratified train-test split.
7. Scale features for model training.
8. Train and compare classification algorithms.
9. Evaluate performance with classification metrics and confusion matrices.
10. Inspect Random Forest feature importance.
11. Perform hyperparameter tuning with GridSearchCV and stratified three-fold cross-validation.
12. Build a prediction function that returns the score, category, and class probabilities.

🤖 Models Compared

- Logistic Regression
- Random Forest Classifier
- Support Vector Machine (SVM)
- K-Nearest Neighbors (KNN)

📊 Results

Results reported by the notebook's final tuning workflow:

Model	**Test Accuracy**	**Weighted F1-score**

Baseline Random Forest	71.2%	0.689
Tuned Random Forest	69.9%	0.683

The baseline Random Forest performed better on this held-out test split. Tuning did not improve the final test metrics, demonstrating why models must be evaluated on unseen data rather than selected only by their cross-validation scores.

🔍 Key Findings

- Alcohol had the strongest positive Pearson correlation with quality in this dataset (approximately 0.485).
- Volatile acidity had a negative Pearson correlation with quality (approximately -0.407).
- Alcohol ranked highest in the baseline Random Forest's feature importance analysis.
- Quality scores 5 and 6 were frequently confused by the models.
- The dataset's quality labels are imbalanced, with most examples belonging to classes 5 and 6.

Correlation and feature importance describe statistical patterns; they do not establish causation.

🏷️ Quality Categories

The prediction function maps scores to categories using these project-defined rules:

- Score ≤ 4: Poor
- Score = 5: Average
- Score = 6: Good
- Score ≥ 7: Excellent

These labels are an interpretation layer added by the project.

🚀 Getting Started

1. Clone the repository

```
git clone YOUR_GITHUB_REPOSITORY_URL
cd YOUR_REPOSITORY_NAME
```

2. Install dependencies

```
pip install numpy pandas matplotlib seaborn scikit-learn jupyter
```

3. Add the dataset

Place `WineQT.csv` in the same directory as the notebook, unless the notebook's data-loading path is updated.

4. Launch Jupyter Notebook

```
jupyter notebook
```

Open `Wine_quality.ipynb` and run the cells in order.

🔮 Future Improvements

- Compare additional classifiers and stronger baselines.
- Improve minority-class evaluation and investigate class imbalance.
- Use nested cross-validation or a separate validation set for model selection.
- Compare predictions against an appropriate ordinal classification or regression approach.
- Add a web interface using Flask or FastAPI.
- Save the trained model and preprocessing pipeline for reuse.
- Deploy a prediction API with input validation and automated tests.

⚠️ Limitations

The reported results are based on one held-out test split. Performance may vary with other splits or datasets. Predicted class probabilities are model estimates and should not be interpreted as guaranteed confidence in real-world wine quality.

👨‍💻 Author

Ashish Singh

GitHub: https://github.com/developerjantaparty

LinkedIn: https://www.linkedin.com/in/ashish-singh-b608a335b

📜 License

Add an appropriate license after confirming the licensing requirements of the dataset and project dependencies.